

RAZONES DE ELECCIÓN DE CÓDIGOS:

Código 1: Hello World

Como primer código se eligió el Hello World, que suele ser el primer código cuando se aprende un nuevo lenguaje, que da una pequeña vista de los comandos básicos del código.

Código

```
section .data
```

```
    msg db "Hello, world!", 0xA
```

```
    len equ $ - msg
```

```
section .text
```

```
    global _start
```

```
_start:
```

```
    mov eax, 4
```

```
    mov ebx, 1
```

```
    mov ecx, msg
```

```
    mov edx, len
```

```
    int 0x80
```

```
    mov eax, 1
```

```
    xor ebx, ebx
```

```
    int 0x80
```

Código 2: Suma

Una de las operaciones básicas para comenzar a ver como son las llamadas al sistema para poder realizar una operación aritmética, el código es muy sencillo por ser uno de los primeros en este ambiente.

Código:

```
section .data
```

```
    buffer db "00", 0xA
```

```
section .text
```

```
    global _start
```

```
_start:
```

```
    mov eax, 5
```

```
    add eax, 10
```

```
    mov ecx, buffer
```

```
    mov ebx, 10
```

```
    xor edx, edx
```

```
    div ebx
```

```
    add al, '0'
```

```
    mov [ecx], al
```

```
    mov al, dl
```

```
    add al, '0'
```

```
    mov [ecx+1], al
```

```
    mov eax, 4
```

```
    mov ebx, 1
```

```
    mov ecx, buffer
```

```
    mov edx, 3
```

```
    int 0x80
```

```
    mov eax, 1
```

```
    xor ebx, ebx
```

```
    int 0x80
```

Código 3: Resta

Una de las 4 operaciones básicas, código que ayuda a seguir practicando como son las llamadas al sistema, imprimir y realizar operaciones aritméticas

Código:

```
section .data
```

```
    msg_result db "Resultado: ", 0
```

```
    len_result equ $ - msg_result
```

```
    newline db 0xA
```

```
section .bss
```

```
    buffer resb 3
```

```
section .text
```

```
    global _start
```

```
_start:
```

```
    mov rax, 25
```

```
    sub rax, 10
```

```
    mov rcx, 10
```

```
    xor rdx, rdx
```

```
    div rcx
```

```
    add al, '0'
```

```
    mov byte [buffer], al
```

```
    add dl, '0'
```

```
    mov byte [buffer+1], dl
```

```
    mov byte [buffer+2], 0xA
```

```
mov rax, 1
mov rdi, 1
mov rsi, msg_result
mov rdx, len_result
syscall
```

```
mov rax, 1
mov rdi, 1
mov rsi, buffer
mov rdx, 3
syscall
```

```
mov rax, 60
xor rdi, rdi
syscall
```

Código 4: Multiplicación

Otra de las operaciones necesarias de saber en cualquier lenguaje es la multiplicación, además nos ayuda a practicar las estructuras y los comandos que se usan en el sistema

Código:

```
section .data
    msg_mul    db "Multiplicacion: ", 0
    len_mul    equ $ - msg_mul
    newline    db 0xA
```

```
section .bss
    buffer resb 16
```

```
section .text
```

global _start

_start:

mov rsi, 20

mov rdi, 5

mov rax, rsi

imul rax, rdi

push rax

mov rax, 1

mov rdi, 1

mov rsi, msg_mul

mov rdx, len_mul

syscall

pop rax

call print_result

mov rax, 1

mov rdi, 1

mov rsi, newline

mov rdx, 1

syscall

mov rax, 60

xor rdi, rdi

syscall

print_result:

```
mov rcx, 10
lea rbx, [buffer+15]
mov byte [rbx], 0
```

.convert_loop:

```
xor rdx, rdx
div rcx
add dl, '0'
dec rbx
mov [rbx], dl
test rax, rax
jnz .convert_loop
```

```
mov rsi, rbx
mov rdx, buffer+16
sub rdx, rbx
```

```
mov rax, 1
mov rdi, 1
syscall
ret
```

Código 5: división

La última de las operaciones básicas en cualquier lenguaje, también necesaria de conocer y de gran ayuda para seguir practicando con programas sencillos

Código:

section .data

```
msg_div db "Division: ", 0
len_div equ $ - msg_div
```

```
newline    db 0xA
```

```
section .bss
```

```
    buffer resb 16
```

```
section .text
```

```
    global _start
```

```
_start:
```

```
    mov rax, 100
```

```
    mov rdi, 5
```

```
    xor rdx, rdx
```

```
    div rdi
```

```
    push rax
```

```
    mov rax, 1
```

```
    mov rdi, 1
```

```
    mov rsi, msg_div
```

```
    mov rdx, len_div
```

```
    syscall
```

```
    pop rax
```

```
    call print_result
```

```
    mov rax, 1
```

```
    mov rdi, 1
```

```
    mov rsi, newline
```

```
    mov rdx, 1
```

```
    syscall
```

```
mov rax, 60
```

```
xor rdi, rdi
```

```
syscall
```

```
print_result:
```

```
mov rcx, 10
```

```
lea rbx, [buffer+15]
```

```
mov byte [rbx], 0
```

```
.convert_loop:
```

```
xor rdx, rdx
```

```
div rcx
```

```
add dl, '0'
```

```
dec rbx
```

```
mov [rbx], dl
```

```
test rax, rax
```

```
jnz .convert_loop
```

```
mov rsi, rbx
```

```
mov rdx, buffer+16
```

```
sub rdx, rbx
```

```
mov rax, 1
```

```
mov rdi, 1
```

```
syscall
```

```
ret
```


Código 6: Comparar mayor, menor o igualdad

Este código se eligió siguiendo un ejemplo de emu, usando el uso de banderas, así como desplegar mensajes a partir de condiciones, comparando registros que contienen números para así determinar si son mayor, o menores o iguales.

Código:

```
section .data
```

```
msg_menor db "El numero en el registro eax es menor que ebx", 0xA
```

```
len_menor equ $ - msg_menor
```

```
msg_igual db "El numero en el registro eax es igual a ebx", 0xA
```

```
len_igual equ $ - msg_igual
```

```
msg_mayor db "El numero en el registro eax es mayor que ebx", 0xA
```

```
len_mayor equ $ - msg_mayor
```

```
section .text
```

```
global _start
```

```
_start:
```

```
mov eax, 10
```

```
mov ebx, 20
```

```
cmp eax, ebx
```

```
jl menor
```

```
jg mayor
```

```
mov eax, 4
```

```
mov ebx, 1
```

```
mov ecx, msg_igual
```

```
mov edx, len_igual
```

```
int 0x80
```

```
jmp salir
```

menor:

```
mov eax, 4
```

```
mov ebx, 1
```

```
mov ecx, msg_menor
```

```
mov edx, len_menor
```

```
int 0x80
```

```
jmp salir
```

mayor:

```
mov eax, 4
```

```
mov ebx, 1
```

```
mov ecx, msg_mayor
```

```
mov edx, len_mayor
```

```
int 0x80
```

salir:

```
mov eax, 1
```

```
xor ebx, ebx
```

```
int 0x80
```

Código 7: Par o impar

Este código se realizó con la finalidad de seguir practicando banderas, parecido al anterior, pero este ayuda a comprobar si el número es par o impar

Código:

```
section .data
```

```
even_msg db "Es par", 0xA
```

```
even_len    equ $ - even_msg  
odd_msg     db "Es impar", 0xA  
odd_len     equ $ - odd_msg
```

```
section .text
```

```
global _start
```

```
_start:
```

```
    mov rax, 17
```

```
    test rax, 1
```

```
    jz .even
```

```
.odd:
```

```
    mov rax, 1
```

```
    mov rdi, 1
```

```
    mov rsi, odd_msg
```

```
    mov rdx, odd_len
```

```
    syscall
```

```
    jmp .exit
```

```
.even:
```

```
    mov rax, 1
```

```
    mov rdi, 1
```

```
    mov rsi, even_msg
```

```
    mov rdx, even_len
```

```
    syscall
```

```
.exit:
```

```
    mov rax, 60
```

```
xor rdi, rdi
```

```
syscall
```

Código 8: Imprimir con loop

Se utilizó loop como ciclo, donde se va incrementando el número que va a imprimir, así como va realizando un salto de line después de cada impresión

Código:

```
section .data
```

```
    newline db 0xA
```

```
section .bss
```

```
    digit resb 1
```

```
section .text
```

```
    global _start
```

```
_start:
```

```
    mov rbx, '0'
```

```
.loop:
```

```
    cmp rbx, '9' + 1
```

```
    jge .end
```

```
    mov [digit], bl
```

```
    mov rax, 1
```

```
    mov rdi, 1
```

```
    mov rsi, digit
```

```
    mov rdx, 1
```

```
    syscall
```

```
mov rax, 1
mov rdi, 1
mov rsi, newline
mov rdx, 1
syscall
```

```
inc bl
jmp .loop
```

```
.end:
```

```
mov rax, 60
xor rdi, rdi
syscall
```

Código 9: Contador de dígitos

El código se le ingresa un número, el cual lo divide en dígitos para así poder contar de cuantos dígitos está compuesto, además se integran partes de códigos anteriores como el loop y la división

Código:

```
section .data
```

```
msg    db "Cantidad de digitos: ", 0
len    equ $ - msg
newline db 0xA
```

```
section .bss
```

```
buffer resb 16
```

```
section .text
```

```
global _start
```

_start:

mov rax, 12345

call count_digits

push rax

mov rax, 1

mov rdi, 1

mov rsi, msg

mov rdx, len

syscall

pop rax

call print_number

mov rax, 1

mov rdi, 1

mov rsi, newline

mov rdx, 1

syscall

mov rax, 60

xor rdi, rdi

syscall

count_digits:

mov rcx, 0

mov rbx, 10

.loop:

```
inc rcx
xor rdx, rdx
div rbx
test rax, rax
jnz .loop
```

```
mov rax, rcx
ret
```

print_number:

```
mov rbx, 10
lea rdi, [buffer+15]
mov byte [rdi], 0
```

.convert:

```
xor rdx, rdx
div rbx
add dl, '0'
dec rdi
mov [rdi], dl
test rax, rax
jnz .convert
```

```
mov rsi, rdi
mov rdx, buffer+16
sub rdx, rdi
```

```
mov rax, 1
```

```
mov rdi, 1
syscall
ret
```

Código 10: Obtención del cuadrado de número

En este código nuevamente se utilizan partes de códigos anteriores, para entender como hacer programas un poco mas complejos usando partes de código muy simple

Código:

```
section .data
```

```
msg db "Cuadrado: ", 0
```

```
len equ $ - msg
```

```
newline db 0xA
```

```
section .bss
```

```
buffer resb 16
```

```
section .text
```

```
global _start
```

```
_start:
```

```
mov r12, 1
```

```
mov rcx, 10
```

```
.print_loop:
```

```
mov rax, r12
```

```
imul rax, r12
```

```
push rcx
```

```
push rax
```



```
mov rax, 1
mov rdi, 1
mov rsi, msg
mov rdx, len
syscall
```

```
pop rax
call print_number
```

```
mov rax, 1
mov rdi, 1
mov rsi, newline
mov rdx, 1
syscall
```

```
pop rcx
inc r12
loop .print_loop
```

```
mov rax, 60
xor rdi, rdi
syscall
```

```
print_number:
    mov rbx, 10
    lea rdi, [buffer+15]
    mov byte [rdi], 0
```

```
.convert:
```

xor rdx, rdx

div rbx

add dl, '0'

dec rdi

mov [rdi], dl

test rax, rax

jnz .convert

mov rsi, rdi

mov rdx, buffer+16

sub rdx, rdi

mov rax, 1

mov rdi, 1

syscall

ret